

Graph Traversals

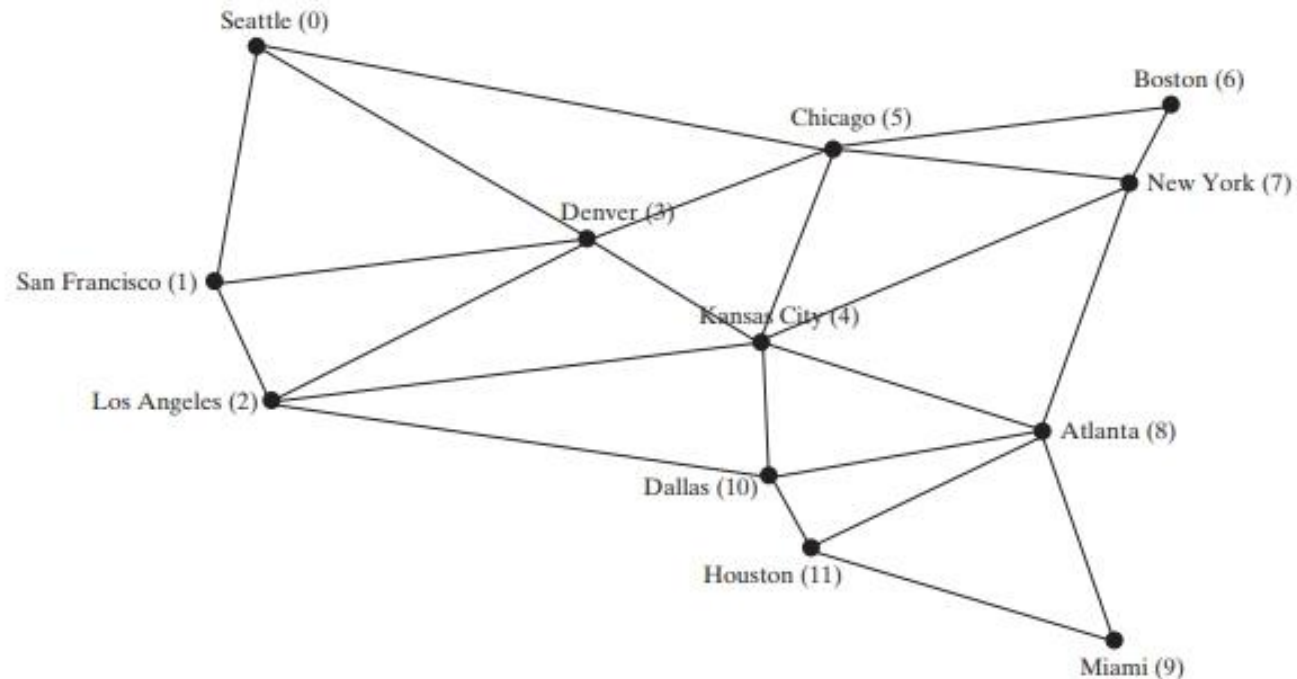
CS 374

Terminology Before Warm-up

- Vertex = an entity
 - Node
 - Plural: vertices
 - All vertices are collected in the set V .
- Edge = an edge (u,v) is present if there is a relationship between the two vertices u and v .
 - All edges are collected in the set E .
- A graph is represented as a pair, $G=(V,E)$.

Warm-Up

- This graph describes direct routes possible between cities.
- With your team, on the board, write a verbal algorithm describing how to visit every vertex of of a graph like this.



Evaluate your own work

1. Does the algorithm describe...
 1. The input & its format?
 2. Where to start?
 3. How to proceed from the starting point, in a way that reasonably covers the whole graph?
2. Is the algorithm **specific** enough? Is there any ambiguity?
3. Are there **errors**? Can you poke holes in the solution?

Category	Points
Efficiency	The proposed solution is completely efficient. There is no possibility of a better implementation.
Completeness	The proposed solution is complete. Every edge case is considered. There are no cases the writer forgot to consider. Given any possible correct input, the output will be produced as described.
Generality	The proposed solution is specified with what counts as valid arguments. It will correctly apply to any given arguments, so long as they are valid.
Specificity	Solution is perfectly specific; there are no questions that should be additionally answered. Transitions are clear. If there are any skipped steps, they are clearly understood to be trivially inferred by the reader.
Conciseness	Writing is perfectly concise. There are no superfluous words and nothing is over-explained.
Presentation	There are no typos in the proposed solution. The layout is not messy. All sentences are complete. Any math or code descriptions present flows cleanly with explanations.

Rotate and evaluate:

- Give each group a score out of 10 for:
- **Efficiency:** __/10
- **Completeness:** __/10
- **Generality:** __/10
- **Specificity:** __/10
- **Conciseness:** __/10
- **Presentation:** __/10

Your homework questions will be evaluated on this scale.

- See table next slide.
- Efficiency, completeness, generality, specificity are emphasized.
- Conciseness and presentation also considered.
- This hinges on your solution being *correct*.

Category	Points	Share of points
Efficiency	The proposed solution is completely efficient. There is no possibility of a better implementation.	20%
Completeness	The proposed solution is complete. Every edge case is considered. There are no cases the writer forgot to consider. Given any possible correct input, the output will be produced as described.	20%
Generality	The proposed solution is specified with what counts as valid arguments. It will correctly apply to any given arguments, so long as they are valid.	20%
Specificity	Solution is perfectly specific; there are no questions that should be additionally answered. Transitions are clear. If there are any skipped steps, they are clearly understood to be trivially inferred by the reader.	20%
Conciseness	Writing is perfectly concise. There are no superfluous words and nothing is over-explained.	10%
Presentation	There are no typos in the proposed solution. The layout is not messy. All sentences are complete. Any math or code descriptions present flows cleanly with explanations.	10%

Do an example HW question

- Imagine you have a closet full of shirts. What can you do to organize your shirts for easy retrieval?

Traversals



What is traversal?

- travel across or through
- move (something) back and forth or sideways
- For graphs: this is equivalent to
 - "searching"
 - "querying"
- We want to record/process/visit every vertex in the graph.

Easy way

- Iterate through the set of vertices
- or
- Iterate through the set of edges

- Pros:
 - $O(V)$ or $O(E)$
 - Professionally, $O(|V|)$ and $O(|E|)$. $|A|$ = the cardinality (size) of set A.
- Cons:
 - Does not take edges/connections into account.

Less Easy Way

- Devise an algorithm to iterate through the entire graph, depending on the edges present.
- Two main algorithms:
 - BFS = Breadth-First-Search
 - DFS = Depth-First-Search

What do you get out of this?

- List of nodes in some order,
 - Like a tree traversal: preorder, inorder, postorder, level order
- A spanning tree
 - A *subgraph* that is a tree, such that every vertex of original graph is included
 - Which may include *back edges* as you go through the algorithm
- The act of processing each vertex in the graph
 - You can attach operations to each visiting step as needed.

Vertex Status

- Every vertex has 1 of 3 statuses:
 - Undiscovered / Unvisited
 - Discovered / Visited
 - Processed

Vertex Status

- Every vertex has 1 of 3 statuses:
 - Undiscovered / Unvisited
 - Discovered / Visited
 - Processed
- Undiscovered / Unvisited: we have not yet arrived at that vertex
- Discovered / Visited: We have arrived at that vertex, but it has not yet been fully processed.
- Processed: We have accomplished all we need to do at this vertex.

BFS

Main Idea

- Start somewhere, add that vertex to a queue
- Until the graph is entirely visited:
 - Dequeue it (mark processed), then enqueue all its neighbors
- Complexity: $O(|V| + |E|)$

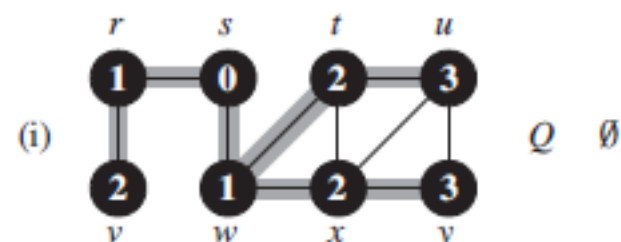
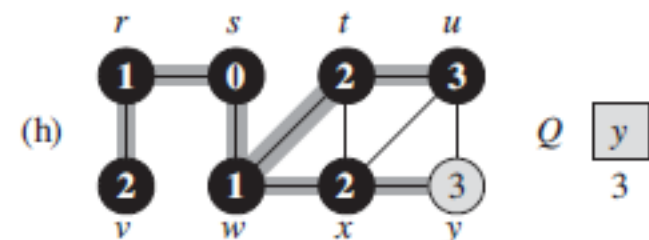
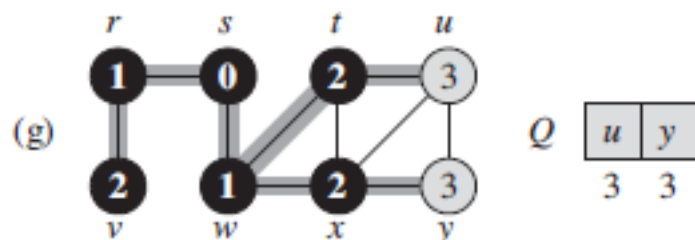
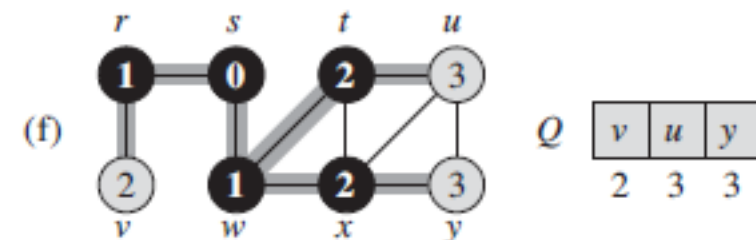
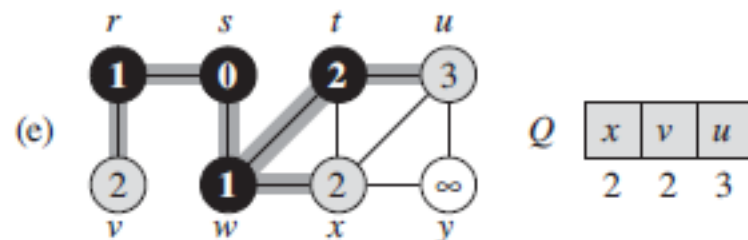
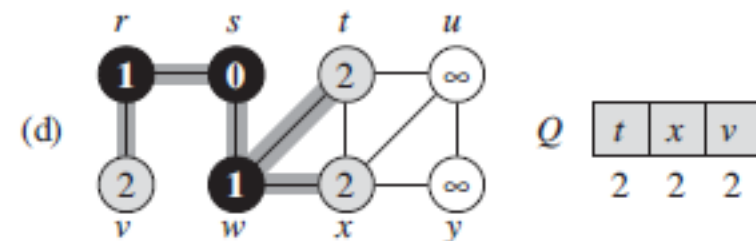
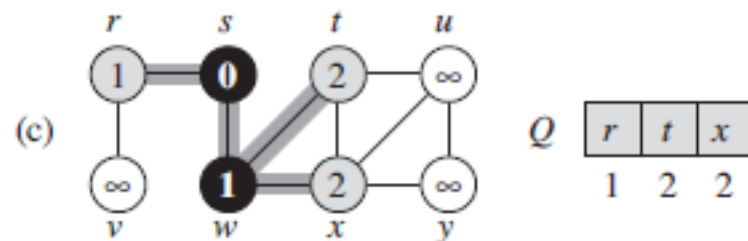
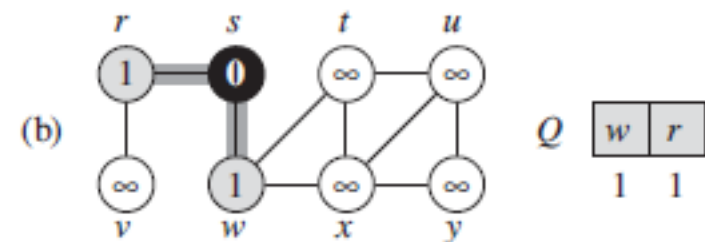
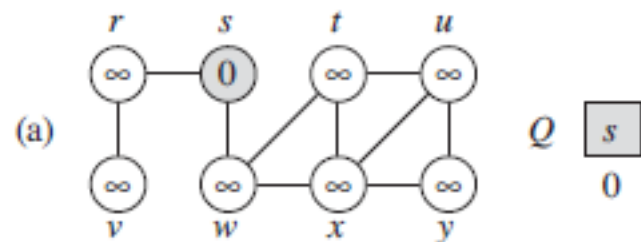
BFS Algorithm v.1

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = \text{WHITE}$ 
3     $u.d = \infty$ 
4     $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = \text{DEQUEUE}(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == \text{WHITE}$ 
14        $v.color = \text{GRAY}$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = \text{BLACK}$ 

```



Insights

- This is *pseudocode* – a sketch of code that won't actually run
 - But it describes pretty much everything we'd need.
- Input is a graph G and a start vertex, s .
- Lots of symbols, ???
- There's a while loop and a queue in there??
- Let's go through it step by step.

BFS(G, s)

Input: G and starting point s .

Properties for each vertex other than s :

- color: white = unvisited
- d : distance
- π : predecessor

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = \text{WHITE}$ 
3     $u.d = \infty$ 
4     $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = \text{DEQUEUE}(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == \text{WHITE}$ 
14        $v.color = \text{GRAY}$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = \text{BLACK}$ 
```

Let's get started. s gets gray color = visited. Distance from s is 0 (we're there). Predecessor is nothing (we start at s).

Q is a (regular) queue, which starts empty. Add s to the queue.

While our queue is not empty, dequeue the latest vertex, u . Look at its adjacent vertices. For each adjacent vertex v :

- If the color is white (unvisited), set it to gray (visited).
- Update its distance +1
- Set its predecessor as u .
- Add v to the queue to be processed.

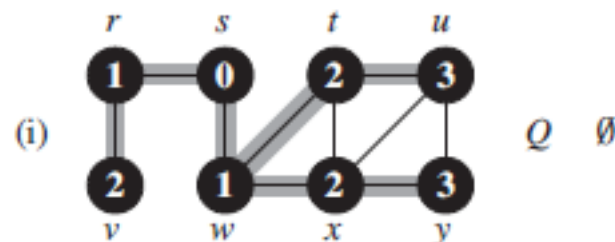
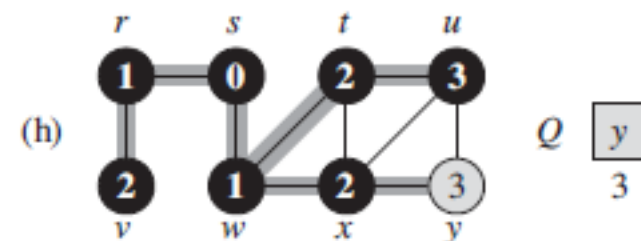
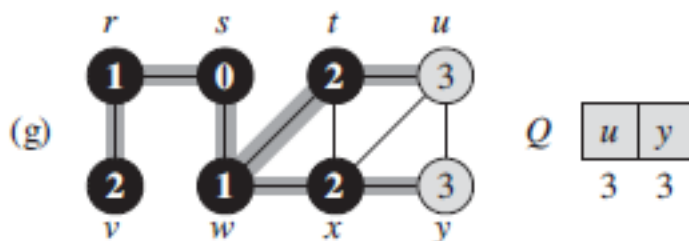
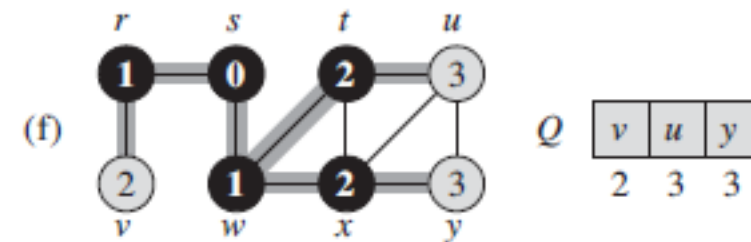
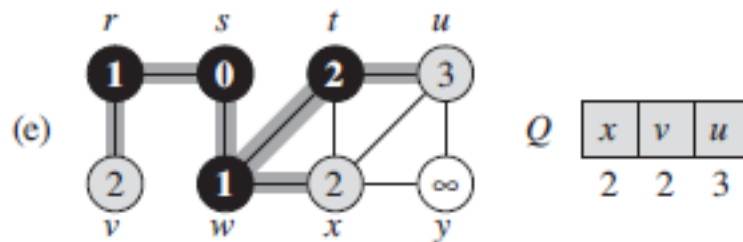
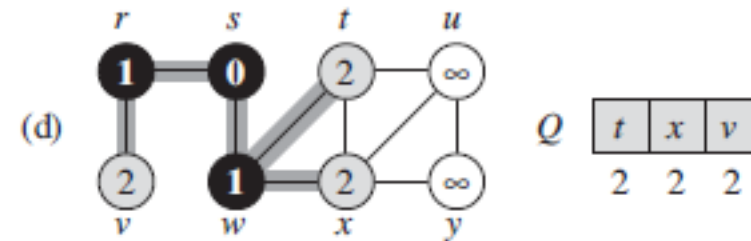
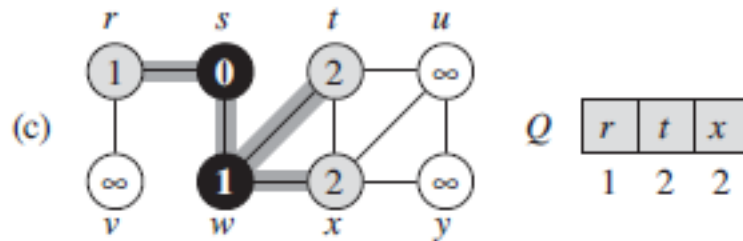
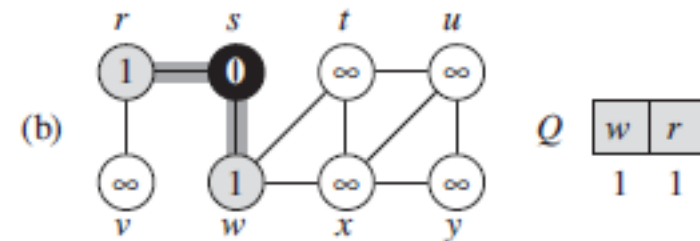
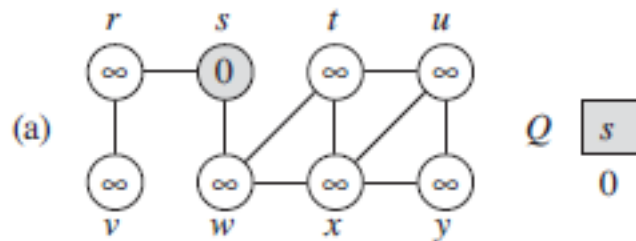
When we have gone through each adjacent vertex, we are done with u . Mark its color as black (processed)

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = \text{WHITE}$ 
3     $u.d = \infty$ 
4     $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = \text{DEQUEUE}(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == \text{WHITE}$ 
14        $v.color = \text{GRAY}$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = \text{BLACK}$ 

```



What do the gray edges mean?

BFS Algorithm v.2

BFS(G, s)

Initialize each vertex $u \in V[G]$ so

$state[u] = \text{"undiscovered"}$

$p[u] = nil$, i.e. no parent is in the BFS tree

$state[s] = \text{"discovered"}$

$Q = \{s\}$

while $Q \neq \emptyset$ do

$u = \text{dequeue}[Q]$

process vertex u if desired

for each vertex v that is adjacent to u

process edge (u, v) if desired

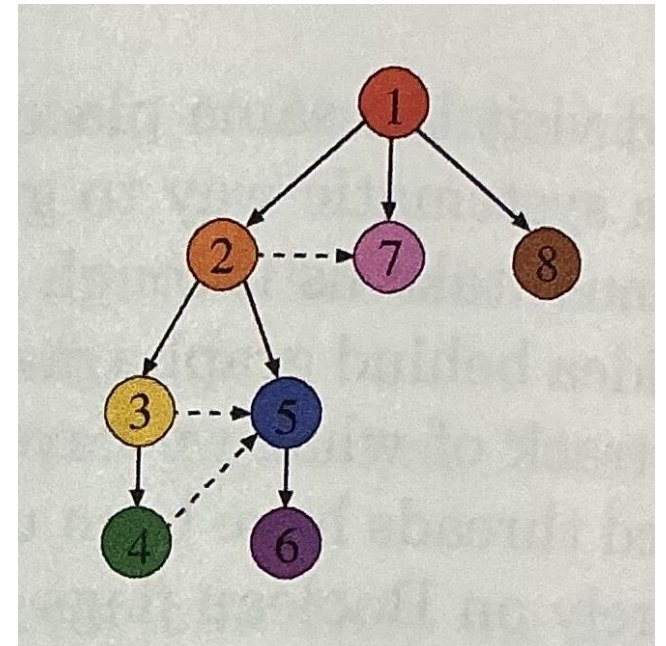
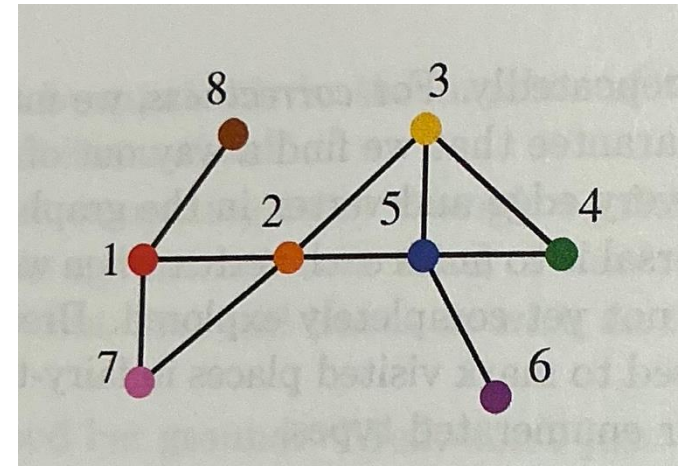
if $state[v] = \text{"undiscovered"}$ then

$state[v] = \text{"discovered"}$

$p[v] = u$

enqueue[Q, v]

$state[u] = \text{"processed"}$



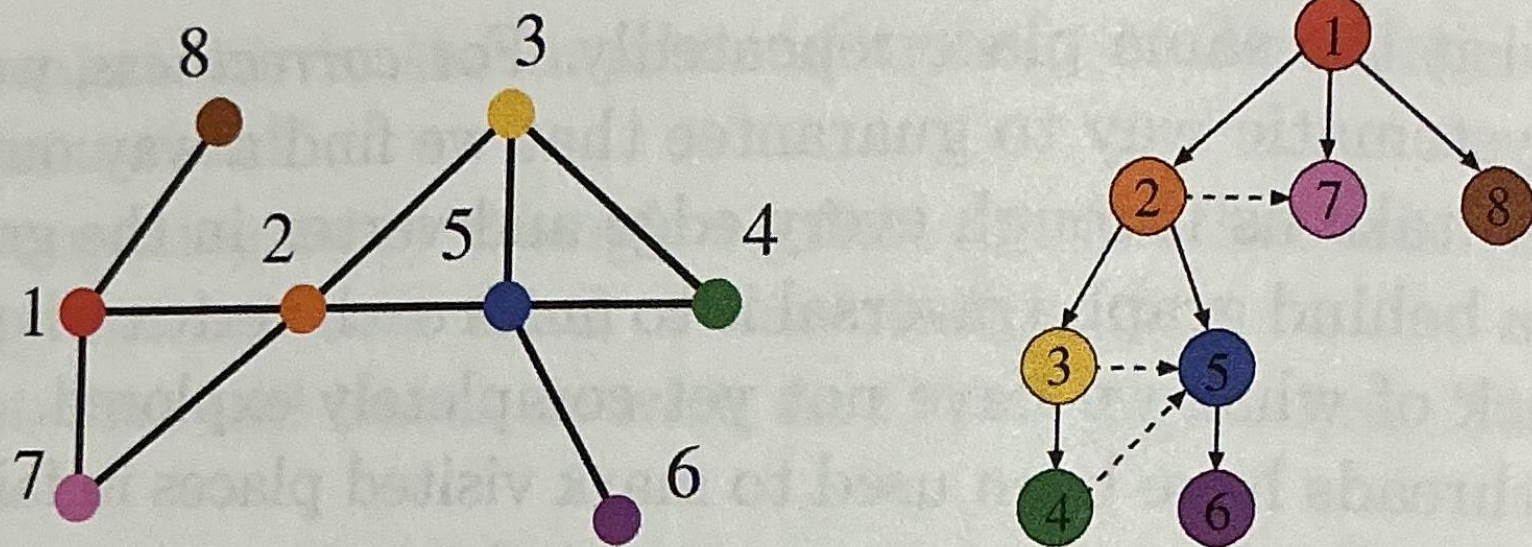


Figure 7.9: An undirected graph and its breadth-first search tree. Dashed lines, which are not part of the tree, show graph edges that go to discovered or processed vertices.

Turn and talk

- What is the difference between the two algorithms?
- In terms of...
 - Notation?
 - The algorithmic process?
 - Intermediate data structures/variables?
 - Result?

BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

BFS(G, s)

Initialize each vertex $u \in V[G]$ so

$state[u] = \text{"undiscovered"}$

$p[u] = \text{nil}$, i.e. no parent is in the BFS tree

$state[s] = \text{"discovered"}$

$Q = \{s\}$

while $Q \neq \emptyset$ do

$u = \text{dequeue}[Q]$

process vertex u if desired

for each vertex v that is adjacent to u

process edge (u, v) if desired

if $state[v] = \text{"undiscovered"}$ then

$state[v] = \text{"discovered"}$

$p[v] = u$

enqueue(Q, v)

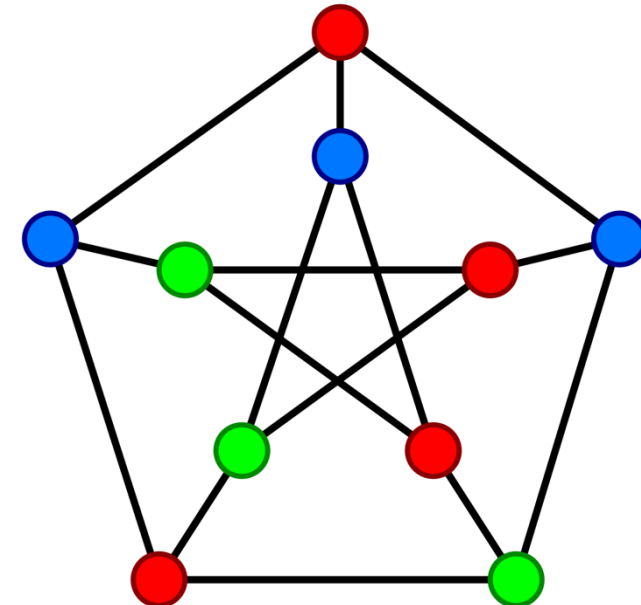
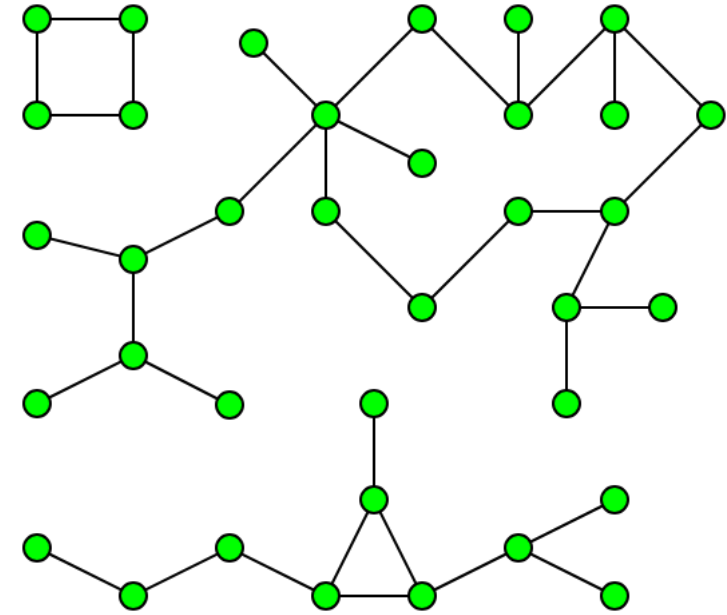
$state[u] = \text{"processed"}$

Do Examples

- Choose a starting node
- Run through the algorithm
- Draw on the board

Benefits of BFS

- Helps find *connected components*
 - Connected parts of the graph
- Adaptations can help with *coloring*
 - Used in scheduling,
 - Register allocation in compilers,
 - Solving sudoku puzzles



DFS

Main Idea

- Start somewhere, add vertex to a stack.
- Until the graph is entirely visited:
 - Pop from the stack, and push all of that node's children (who have not been visited)
- Focus on children before neighbors: Depth before breadth
- Complexity: $O(|V| + |E|)$

DFS Algorithm v.1

DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$                                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$                             // explore edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$                                 // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
```

Insights

- Two algorithms this time instead of one.
 - One is recursive.
- Similar colors/predecessor notation as previously.
- But there's other information too. f, time. ????
- Let's go through it step by step.

DFS(G)

Input: G in general.

Properties for every vertex:

- color: white = unvisited
- π : predecessor

1 **for** each vertex $u \in G.V$

2 $u.color = \text{WHITE}$

3 $u.\pi = \text{NIL}$

4 $time = 0$

5 **for** each vertex $u \in G.V$

6 **if** $u.color == \text{WHITE}$

7 DFS-VISIT(G, u)

Start the "timer" (# steps/distance from starting point) at 0.
For each vertex, if it is white (unvisited), run DFS-VISIT() on it.

DFS-VISIT(G, u)

1 $time = time + 1$

2 $u.d = time$

3 $u.color = \text{GRAY}$

4 **for** each $v \in G.Adj[u]$

5 **if** $v.color == \text{WHITE}$

6 $v.\pi = u$

7 DFS-VISIT(G, v)

8 $u.color = \text{BLACK}$

9 $time = time + 1$

10 $u.f = time$

We discovered u , so add 1 to time, update u 's distance, mark as gray (visited)

Look at each vertex adjacent to u . Visit it recursively.

We're done with u ! Mark it processed.

DFS(G)

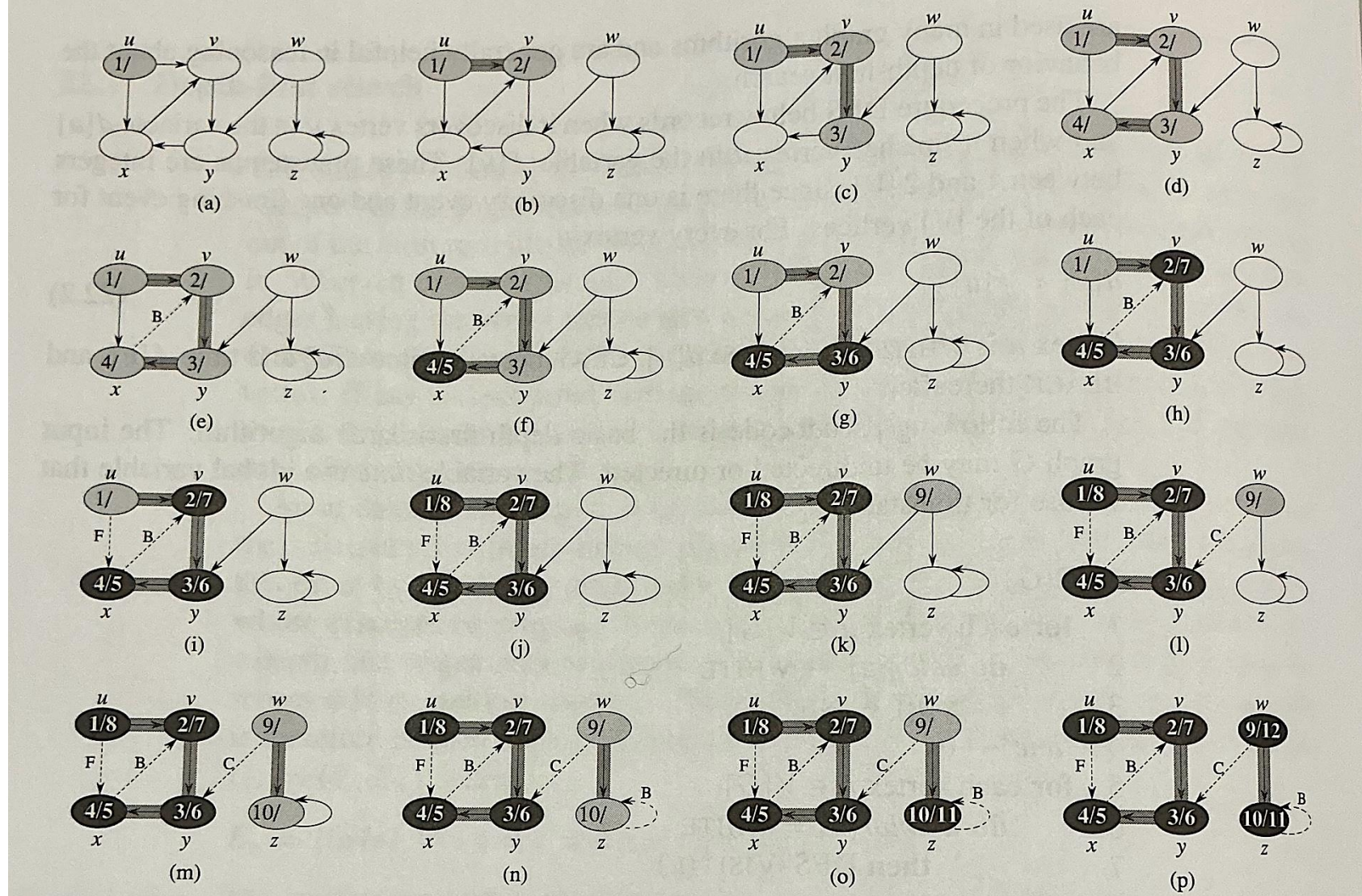
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
    
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```



Could we run this algorithm on an undirected graph?

Types of edges

- Tree edges
 - (u,v) is a tree edge if v was first discovered by exploring edge (u,v)
- F = Forward edges
 - We could have gone "forward" in the algorithm to this node, but we've already processed it
 - Nontree edges (u,v) connecting u to a descendant v in a depth-first tree.
- B = Back edges
 - Edges (u,v) connecting a vertex u to an ancestor v in a depth-first tree
 - Self-loops are considered back edges.
- C = Cross edges
 - All other non-included edges.

DFS Algorithm v.2

DFS(G, u)

$state[u] = \text{"discovered"}$

process vertex u if desired

$time = time + 1$

$entry[u] = time$

for each vertex v that is adjacent to u

process edge (u, v) if desired

if $state[v] = \text{"undiscovered"}$ then

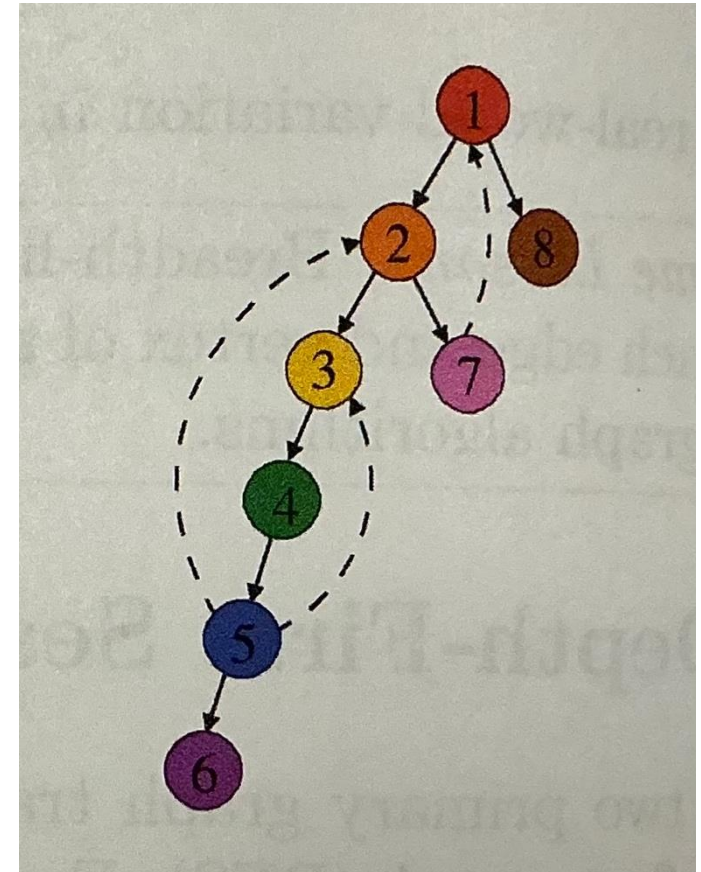
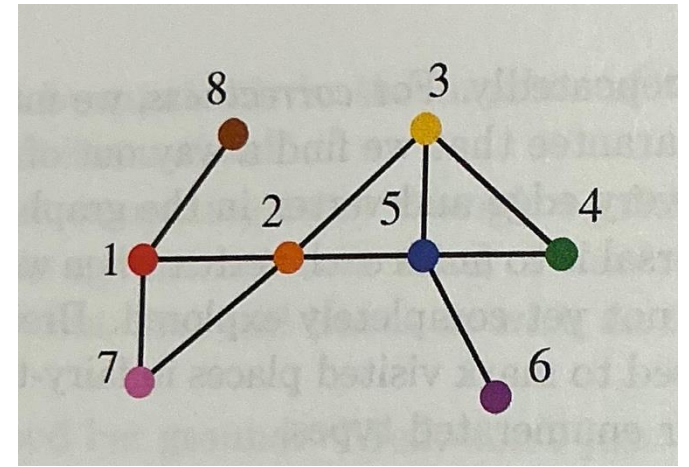
$p[v] = u$

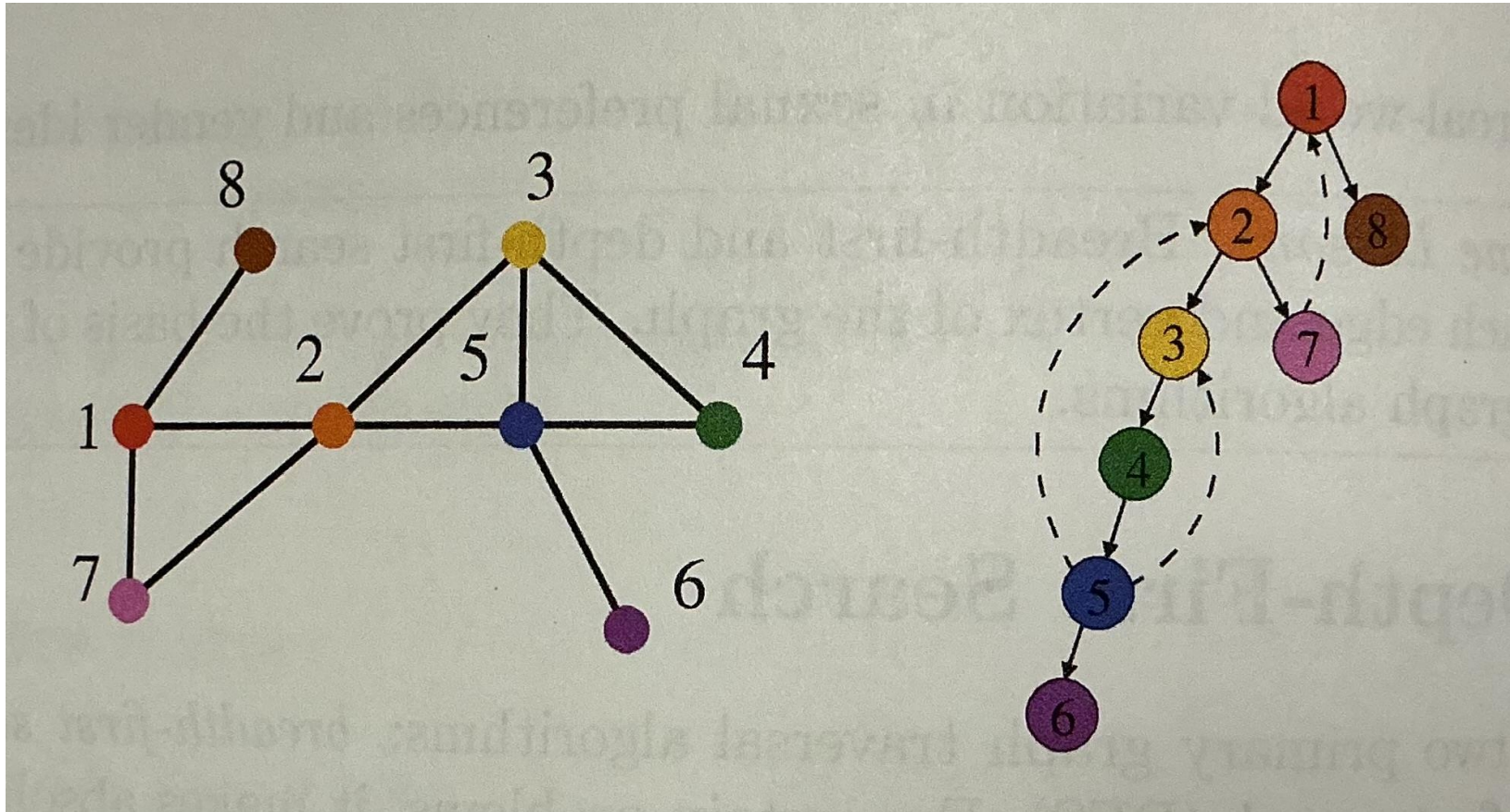
DFS(G, v)

$state[u] = \text{"processed"}$

$exit[u] = time$

$time = time + 1$





Dashed lines, which are not part of the tree, indicate back edges.

Turn and talk

- What is the difference between the two DFS algorithms?
- In terms of...
 - Notation?
 - The algorithmic process?
 - Intermediate data structures/variables?
 - Result?
- BFS and DFS are similar algorithms. What distinguishes BFS from DFS?

DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

DFS(G, u)

```
 $state[u] = \text{"discovered"}$ 
process vertex  $u$  if desired
 $time = time + 1$ 
 $entry[u] = time$ 
for each vertex  $v$  that is adjacent to  $u$ 
    process edge  $(u, v)$  if desired
    if  $state[v] = \text{"undiscovered"}$  then
         $p[v] = u$ 
        DFS( $G, v$ )
 $state[u] = \text{"processed"}$ 
 $exit[u] = time$ 
 $time = time + 1$ 
```

Do Examples

- Choose a starting node
- Run through the algorithm
- Draw on the board

Benefits of DFS

- Helps find cycles (back-edges)
- Testing connectivity
- Topological sorting (on directed, acyclic graphs)

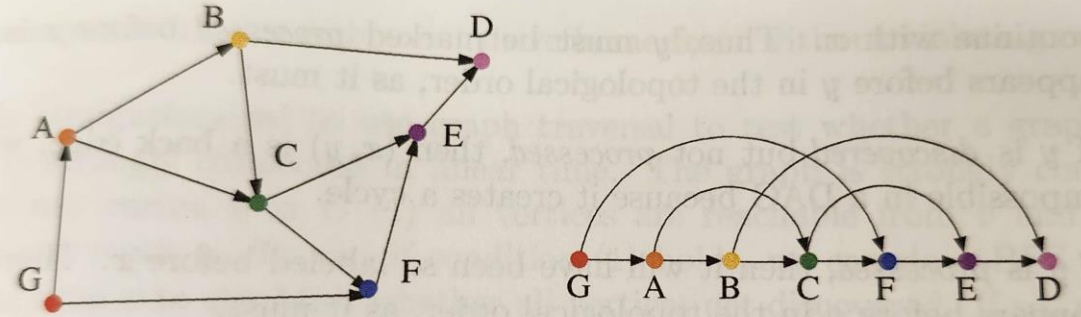


Figure 7.15: A DAG with only one topological sort (G, A, B, C, F, E, D)

Discuss

- What did you learn from comparing different formulations of the same algorithm? (e.g. BFS – BFS; DFS – DFS)
- What questions do you still have about how BFS and/or DFS work?

Cool-Down

- Describe a modification to the data structure or algorithm that answers the question:
 - How can you use Union-Find to compute the number of connected components?
 - How can you use BFS to compute the number of connected components?
 - How can you use DFS to compute the number of connected components?